High-Level

CORE Platform

Client-Side

Core Tech	React 18, TypeScript, Vite.
State & Data	TanStack Query handles server state (caching/fetching), while Wouter manages routing
UI/UX	Styled with Shadcn/ui (Tailwind CSS). The interface is department-scoped (Sales, Purchasing, Accounting) with an "Action Needed Strip" for proactive task management.
Client-Side AI	Local inference using Hugging Face Transformers (@xenova) for tasks like PDF parsing and text-to-speech without external API costs.

Application Logic

Runtime	Node.js with Express.js.
Security & Multi-Tenancy	Replit Auth (OIDC) manages logins
AI Engine (Atlas AI)	unified `aiClient` abstraction layer that routes requests through department-scoped profiles, enforcing RBAC (Role-Based Access Control).
Business Rules Engine	Manages vendor selection logic (e.g., `lowest_cost_instock`) and GP floor violation workflows.

Data & Persistence Layer

ORM	**Drizzle ORM** (TypeScript-first) manages the schema and migrations
Database	Before: Neon PostgreSQL (Replit Managed) Now: PostgreSQL (on DigitalOcean server) scalable storage for tenants, quotes, orders, and user preferences.
Encryption	Sensitive data (like supplier credentials and OAuth tokens) is protected via AES-256-GCM.

Integration & Sync Hub

Category	**Integration**	**Purpose**
ERP	Odoo vs ERPNext	System of record for accounting, payments, and full lifecycle sync.
Distributors	D&H, Ingram Micro, TD SYNNEX	Real-time pricing, stock availability, and automated order submission.
Communication	Gmail, Outlook	Marketing Hub email campaigns and tracking.
Automation	Local Transformers	PDF/Excel parsing for RFQ imports and internal pricing updates.

ERP as integration

Can ERPNext/Odoo act as a temporary, free, accounting microservice integrated into a Node.js SaaS, and be removable later?

Clarification:

- ERPNext should not own or share the Postgres
  - Still need database specific schemas, APIs, etc
- It should be a microservices style, replaceable accounting service used to provide features for now
- Stripe needs to sit on Nodejs. Node handles the logic and triggers ERPN actions

Proposed architecture:

Client on Frontend
↓
Node.js --> Postgres(DB) System data
| For account events
| REST / Webhooks / Jobs
v
ERPNext
(Accounting + Invoicing + Reports)

Component	What it does	Tech	How
Node.js	Main Backend - Manage and auth users - Workflow and Business rules - Stripe logic - Subscription state - Pricing logic - Order lifecycle	JS	
Database	Stores: Users, Roles, Supcriptions	Postgres	
ERPNext	Accounting Invoices Bills Payments Chart of Accounts Tax rules Basic Financial Reports - Doesn't own data	Python (Frappe)	Self-host on droplet Communicate with Backend using REST + hooks. Basic Stripe

ERP Architecture

- Use ERPNext as temporary, free, accounting-only, one day replaceable service
- If ERPNext is down, app should still work with some services unnaivailable
- Create 1 ERP “Company” per CompanyAccount
- Create 1 ERP service service user per Company (not individual users)
- When a user from that account is connected, Node authenticates ERP with the service user, so any of its features can be used.
- ERP UI is not shown or used by users

Invoices:

- When a user or the system wants to generate an invoice, it creates this instance in the database, it calls ERP’s Sales Invoice API to generate it,and returns an erp_invoice_id, node uses it for reference, and database stores.
- A request to view past invoices will query the postgres, or ERPNext.
  - Endpoint: `https://{your.erpnext.instance.com}/api/resource/Sales%20Invoice`

Stripe

- Stripe is managed by Node
- ERPNext can record payments, attach references, tec.

User stories:

Core accounting

- When the invoice is finalized, it’s recorded in ERP for accounting
- As accounting, I want to see AR aging by company.
- As accounting, I want tax-compliant reports without custom code.

Purchase/vendor flows

- When a distributor order is placed, create a bill in ERP.
- When a vendor invoice arrives, match it to orders.
- Track AP aging per supplier

Stripe reconciliation

- When Stripe marks an invoice as paid, ERP records the payment.
- When refunds occur, ERP records credit notes
- ERP reports match Stripe totals (but Stripe remains source).

Events sent to ERP:

- Invoice Created, InvoicePaid, BillReceived, RefundIssued

Questions:

- Who is the system of record for invoices?
  - PostgresDB. But when ERPNext is used does it own it?
  - Where does an invoice first come into existence?
  - If ERP and Postgres disagree, which one wins?
- What data must *only* ever live in Postgres?
  - Users, Accounts, Subscriptions, etc
- Can ERP be down without breaking the app?
  - Will require logic
  - What functionalities are unavailable when ERP is down?
- Can we recreate ERP data from Node events?
  - Later in app lifecycle
- How do we delete ERP later?
  - How long would a migration away from ERP take?
  - What ERP data must be exported before shutdown?

- What Stripe logic must not live in ERP?
  - Who handles Stripe webhooks?

- What tables must never be duplicated across systems?
- What are the exact API contracts between Node and ERP?
- ERP maintenance and upgrades?
- When do we replace ERP? What would be a “red flag” as we implement ERPNext?

Resources:

- ERPNext REST API docs
- Frappe permissions & roles
- ERPNext accounting module overview
- ERPNext Stripe integration docs
- ERPNext multi-company setup

ERP as backend

Clarification:
- ERPNext is the primary database
- Manages authentication authority
- Authorization engine
- Business object ownter (Users, Accounts, Subscriptions, Invoices, etc)

Proposed architecture:

Client (React + Vite)
↓ HTTPS (JWT / Session)
Server (Node/Express)
| For account events
| REST
v
ERPNext → Maria DB
(owns everything)

Component	What it does	Tech	How
React UI	Frontend UI for CORE app	TS	
Node.js	Main Backend - APIs - Security - Validation - Stripe webhooks - May replace ERP later	JS	
ERPNext	Stores: Users, Roles, Supcriptions Owns: - Users - Company/Account/Subscriptions - Plans - Accounting - Invoices - Bills - Payments - Chart of Accounts - Tax rules - Basic Financial Reports	Python (Frappe)	Self-host on droplet Communicate with Backend using REST + hooks. Basic Stripe

Stripe integration lives in node. Node validates and calls ERP API
- React should not talk to ERP

To do:

On

1. Create Custom Doctypes
2. Define fields
3. Define permissions
4. Define workflows / relationships (using links)
5. Write Frappe server scripts, custom ERP code
6. Node authentication with ERP
    a. User logs in with ERP API
    b. ERP returns session cookie
    c. Node forwards cookie to client which stores it
7. OR Node authentication with JWT

Directory

cd erpbackend

Inside /server:

/server
  /routes
  /controllers
  /services
    erpService.ts ERP service module (handles auth, api calls, etc)
  /middleware
    auth.ts

Define:

- Does ERP manage password reset?
- Does ERP manage email verification?
- Does Node mirror any user state?
- How does multi-tenancy work?
- Will each customer be a separate ERP Company? Or a single ERP instance with tenant field?

Migrate postgres data

- Extract existing data
- Transform into ERP schema
- Import via ERP API or data import tool

Building minoria_backend app for erp github:

Installing the app on the site

```
bench --site 174.138.95.56 install-app minoria_backend
bench migrate
bench build --force
bench restart
```

- https://1a6ca735-6be6-41ba-9aad-7e2c529ec2f8-00-2ztperoarwhh.spock.replit.dev/dashboard/admin/storesite
-

Questions:
- Security: multi-database on erp next? Separate DB for users/account info? Single database for all data?
- ERPNext questions

ERPNext is the backbone

In your model, ERPNext is the system of record for operational business objects across departments (CRM → Sales → Orders → Invoicing → Accounting)

CORE (React/Node) is the experience layer + integration/orchestration layer that sits on top.

Corrected System Roles (ERPNext as Backbone)
1) ERPNext (Server hosted on DigitalOcean) = The Backbone / Primary Business System
ERPNext owns and manages:
- CRM (Leads, Opportunities, Customers)
- Sales (Quotes, Sales Orders)
- Fulfillment / Order workflow (as designed in ERPNext)
- Invoicing
- Accounting / GL / AR/AP
- Inventory/Item masters if you choose to keep it there

It's the operational ERP for departments.

2) CORE Front-End (coreSG.net and coreSMB / React) = Customer/Reseller UI Look & Feel
This is the customer-facing experience (storefront, procurement portal UX).
it's the UI layer that connects to the server to retrieve ERPNext data.

3) CORE Backend (Node) = Secure API Gateway + Integrations + UX Orchestration
This layer exists to:

- Provide a secure API facade (don't expose ERPNext directly to customers)
- Handle RBAC in your CORE-defined way (or map it to ERP roles)
- Run distributor integrations (D&H / Ingram / TD SYNNEX)
- Normalize catalog/pricing/availability and push results into ERP workflows
- Handle webhooks, retries, idempotency, logging
- Potentially handle Stripe (if Stripe is your billing engine) while ERPNext handles invoicing/accounting entries

So ERPNext is the backbone, but CORE is the customer portal + integration brain.

Customer UI (CORE)
→ calls CORE Node API
→ which reads/writes to ERPNext (CRM/Sales/Invoice/etc.)
→ while CORE Node also calls distributors and feeds data back into ERPNext objects (items, quotations, order lines, etc.)
ERPNext becomes the single source of truth for departmental objects.

The Real Integration Challenge (Your Actual Pain Point)
When ERPNext is backbone, the hard part isn't "logging in." It's:

- Which objects live in ERPNext vs CORE DB
- How users/auth work (ERP auth vs CORE auth vs SSO)
- Role mapping (ERP roles ↔ CORE RBAC)
- How distributor real-time pricing becomes ERP quotes
- Document workflow mapping (Opportunity → Quotation → Sales Order → Invoice)
- Webhooks + retry strategy so nothing double-posts

That's what the team is "deciphering" — and it's normal.

Message to Sameer (Corrected, Ready to Send)
Sameer —
Quick clarification so we're aligned on system roles:
ERPNext on DigitalOcean is the backbone of the platform. It's not just accounting — it's intended to run the departments end-to-end: CRM + Sales + Orders + Invoicing + Accounting + possibly users (and related workflows).
coreSG.net is the customer-facing storefront look/feel we want. It's the UI experience layer, not the system of record.
coreSMB / the Node layer is our secure API + integration layer that sits between the UI and ERPNext:

- enforces our RBAC patterns (or maps them to ERP roles)
- connects to distributors (D&H / Ingram / TD SYNNEX)
- orchestrates quote-to-order flows
- handles webhooks/retries/logging
- and pushes/pulls the needed objects in ERPNext

So the direction is:
Customer UI → CORE API layer → ERPNext (backbone)
The current work is deciphering the integration boundaries and the exact data/workflow mapping so all objects (CRM → Sales → Invoice → Accounting) flow cleanly through all components.

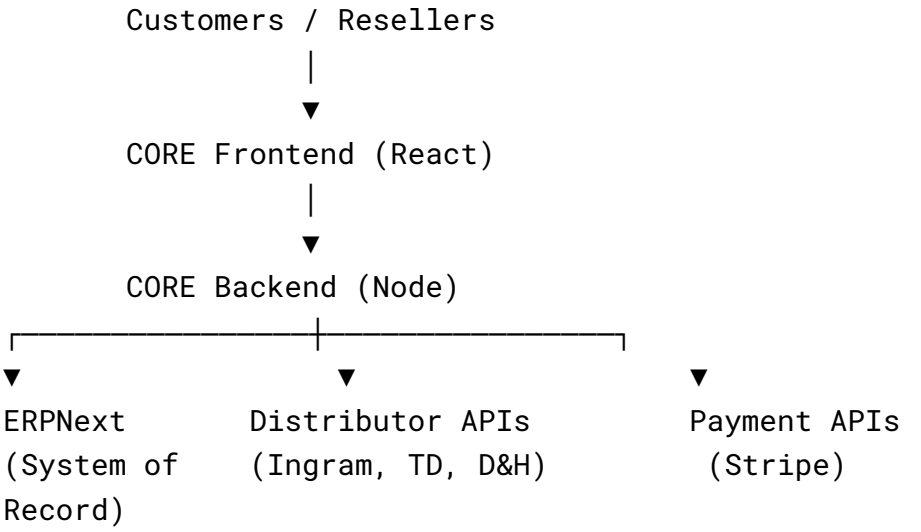
Email:

Hi Sameer,

This is an explanation on where the project currently stands and how the system architecture is designed, also to get your advice on a few technical and application shipping questions and we move forward.

The platform is intended to be a SaaS business operations platform where small businesses or resellers can have tools such as CRM, quoting, purchasing, order flow, accounting, marketing, and reporting into a single system to reduce cost. We want to create end-to-end workflows, reduce usage for multiple tools at once such as (Trello, QuickBooks, or CRMs), have different departments and users in one platform, and integrate AI into workflow automations.

Current Architecture:



1. CORE Frontend (React): This is the customer/reseller client interface.
   The UI handles: the storefront for CORE and subscription management, customer dashboard, product configuration, quotes, invoices, CRM, and purchasing actions
   It communicates directly with the backend APIs, not ERPNext directly.

2. CORE Backend (Node):
   Node backend sits between the frontend and ERPNext
   It works to: be a secure API gateway between the two services, enforce rules and map to ERPNext roles, integrate with external APIs and distributors (D&H / Ingram / TD SYNNEX), implement quote and order workflows, request AI automations, handle login and authentication with ERP, and manage payment integrations like Stripe.
   The flow would be: Customer UI –> CORE Node Backend → ERPNext

3. ERPNext
   - It is currently hosted on a digitalocean droplet

   ERPNext is intended to manage the business objects and workflows, including: account and user management, CRM data, order fulfilling, invoicing, accounting, and possibly inventory and items.

Current Direction: ERPNExt

The direction we're moving to is using ERPNext as the operational backbone and data management of the platform, with a UI application that integrates with it. The idea is to have ERPNext as the system of record for user data and business operations across departments, while the CORE application stack handles the customer-facing client and integrations behind the scenes.

Why ERP:

- We chose ERPNext as the backbone for it's open-source usage. It already models the CRM → sales → order → invoice → accounting cycle
- It provides the workflow without the need to build systems from scratch
- It allows for faster product shipment

Where things live:

ERPNext: is running on a DigitalOcean droplet

- ERPNext Admin interface: The ERPNext admin UI is accessible through the server and is being used right now to migrate data from the Postgres database into this system.

- Accessed here: http://174.138.95.56:8080/#login using Administrator@admin.com and @2026!Minori4Tech!1nterns

Code repositories: CORE frontend/backend code is currently on Github

Application
The React frontend can be seen here: https://coresmb.com/
The storefront for businesses can be seen here: https://coresg.net/

Our current organization is on a shared Drive: Minoria Tech Interns

Where we still need to decide the right approach:
- Is there a benefit from having objects live in a CORE database vs ERPNext
- How should authentication work, and role mapping work?
- How document workflows occur and how we can build these automations across systems (Opportunity → Quote → Sales Order → Invoice)

Data Architecture questions:
- Should we keep everything inside a single ERPNext database, or split data across multiple databases? Separate database for customer accounts/authentication and keep ERPNext focused solely on operational business objects?
- If on ERPNext, would it make sense to assign a user to each company, with its own database, or hold everything under a single user?
- How would you recommend structuring authentication between the CORE platform and ERPNext? What are the security implications for these kinds of applications

Product process questions:
- A challenge has been helping interns be productive with the codebase while some are getting comfortable with Git workflows still. We want to structure development so we keep making progress while the project moves on.
- How would you advice to structure repos for a project like this?
- How about managing interns contributing to a shared architecture? Making changes to production such as the admin account for ERPNext UI, or using agile backlogs to help development?

Thanks again for your time.

Best,

Joe